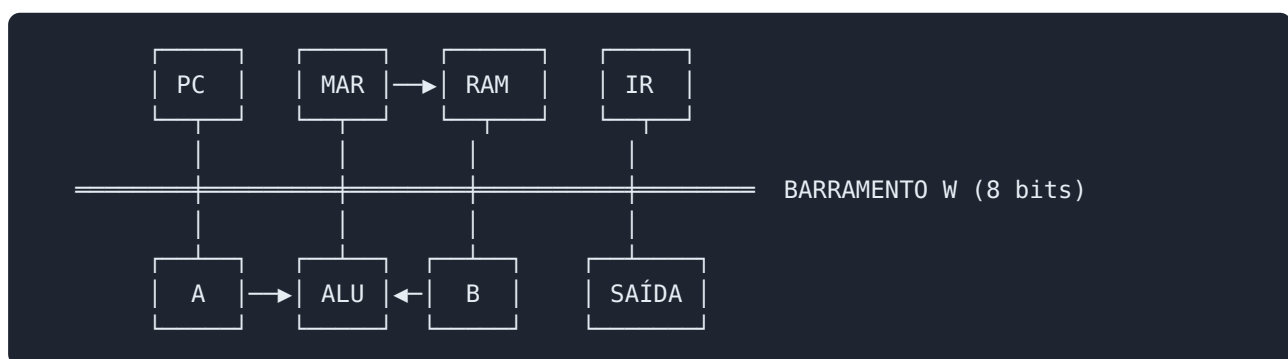


Componentes do SAP-1 — explicação detalhada

Guia de estudo de **cada bloco** do processador SAP-1, com o código e o papel de cada um. Serve de referência para entender o projeto módulo a módulo.

Visão geral (o datapath)

Todos os blocos conversam por um único fio de 8 bits: o **barramento W**. Em cada estado (T1-T6), o controlador liga **um** bloco para "falar" no barramento e **um ou mais** para "ouvir".



O "padrão registrador" (leia isto primeiro)

Cinco blocos — **MAR, IR, Acumulador A, Registrador B e Registrador de Saída** — são o **mesmo circuito**: um registrador que **carrega do barramento quando habilitado** e **zera no reset**. Muda só a largura e o nome do sinal de carga.

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)           // reset assíncrono (ativo em baixo)
        q <= 0;             // zera
    else if (!load_bar)      // sinal de carga ativo (em baixo)
        q <= bus_in;        // copia o barramento
    // senão: mantém o valor (não faz nada)
end
```

Três comportamentos, em ordem de prioridade: 1. **Reset** (`clr_bar=0`): zera na hora, sem esperar o clock. 2. **Carrega** (`load_bar=0`): na subida do clock, copia o barramento. 3. **Segura**: se nenhum dos dois, mantém o valor guardado.

"Ativo em baixo" (o til `~` no nome, ex.: `~La`) significa que o sinal **age quando vale 0**. É a convenção do SAP-1 para os sinais de carga.

Agora, cada bloco em detalhe.

1. Program Counter (PC) — `program_counter.v`

O que é: um **contador de 4 bits** que guarda o endereço da **próxima instrução**. Vai de 0 a 15 e volta a 0.

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar) pc_out <= 4'b0000;    // reset -> 0
    else if (cp)   pc_out <= pc_out + 1; // Cp ativo -> incrementa
end
```

- **cp** (Cp): quando 1, o PC **incrementa** na próxima subida do clock. Diferente dos registradores, o PC não "carrega do barramento" — ele só **conta**.
- No **T2** de toda instrução o **cp** fica ativo → o PC já aponta para a instrução seguinte antes mesmo de executar a atual.
- É de **4 bits** porque a memória só tem 16 posições (0-15).

Papel: dizer "qual é a próxima instrução". No T1, o valor do PC é copiado para o MAR para buscar essa instrução.

2. MAR (Memory Address Register) — `mar.v`

O que é: um registrador de **4 bits** que guarda o **endereço que a RAM vai ler**. É o "dedo apontando" para uma posição da memória.

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)   addr_out <= 4'b0000;
    else if (!lm_bar) addr_out <= bus_in; // ~Lm ativo -> carrega
end
```

- **lm_bar** (~Lm): quando 0, o MAR **carrega** os 4 bits que estão no barramento.
- A saída **addr_out** vai direto para a entrada **addr** da RAM.

Por que existe? A RAM precisa de um endereço **estável** para ler. O MAR "segura" esse endereço enquanto a RAM entrega o dado. Ele é carregado duas vezes por instrução: - **T1**: recebe o PC (para buscar a **instrução**). - **T4**: recebe o operando do IR (para buscar o **dado**).

3. Instruction Register (IR) — `instruction_register.v`

O que é: guarda a **instrução** de 8 bits lida da RAM e a **divide em dois pedaços de 4 bits**.

```
reg [7:0] ir;
always @(posedge clk or negedge clr_bar) begin
```

```

    if (!clr_bar)      ir <= 8'b0;
    else if (!li_bar)  ir <= bus_in;    // ~Li ativo -> carrega a instrução
end
assign opcode = ir[7:4];    // nibble alto = qual instrução
assign operand = ir[3:0];  // nibble baixo = endereço do operando

```

- **li_bar** (~Li): quando 0, carrega a instrução (feito no **T3**).
- **opcode** (bits 7-4): diz **o que fazer** (LDA/ADD/SUB/OUT/HLT). Vai para o **controlador**.
- **operand** (bits 3-0): diz **onde está o dado** (um endereço). Vai para o barramento (via **~Ei**) e daí para o MAR no T4.

Papel: é a "instrução em execução". O controlador olha o **opcode** para decidir os próximos passos (T4-T6).

4. Acumulador A — `accumulator.v`

O que é: o registrador de **8 bits** mais importante — é **onde a conta acontece**. Quase toda instrução mexe nele.

```

always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)      acc_out <= 8'b0;
    else if (!la_bar)  acc_out <= bus_in;    // ~La ativo -> carrega A
end

```

- **la_bar** (~La): quando 0, A carrega o valor do barramento.
- A saída **acc_out** faz **duas coisas ao mesmo tempo**: 1. alimenta **continuamente** a ALU (entrada **acc**); 2. vai para o barramento quando **Ea** está ativo (ex.: no **OUT**).

Quem escreve em A? - **LDA**: $A \leftarrow \text{dado da RAM (no T5)}$. - **ADD / SUB**: $A \leftarrow \text{resultado da ALU (no T6)}$.

É por isso que, no 3×4, você vê **acc_out** indo $3 \rightarrow 6 \rightarrow 9 \rightarrow 12$.

5. Registrador B — `register_b.v`

O que é: registrador de **8 bits** que guarda o **segundo operando** da soma/subtração. A ALU sempre calcula $A \pm B$, então B precisa segurar o outro número.

```

always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)      b_out <= 8'b0;
    else if (!lb_bar)  b_out <= bus_in;    // ~Lb ativo -> carrega B
end

```

- **lb_bar** (~Lb): quando 0, carrega B (feito no **T5** de ADD/SUB).
- A saída **b_out** alimenta a ALU continuamente (entrada **breg**).

Papel: no **ADD 11**, o T5 carrega B com o dado (3); no T6 a ALU faz $A + B$ e o resultado volta para A.

6. ALU (somador/subtrator) — `adder_subtractor.v`

O que é: a **calculadora**. É **combinacional** (sem clock): calcula na hora, o tempo todo.

```
assign result = su ? (acc - breg) : (acc + breg);
```

- `su` (Su): seletor da operação.
- `su = 0` → `result = A + B` (soma)
- `su = 1` → `result = A - B` (subtração, por complemento de dois)
- As entradas `acc` e `breg` vêm direto dos registradores A e B.
- A saída `result` vai para o barramento quando `Eu` está ativo (no T6).

Importante: a ALU **não guarda nada**. Ela sempre mostra `A ± B` do momento. Por isso, na onda, `result` fica "mexendo" conforme A e B mudam — é normal; o que importa é o valor no instante em que `Eu` joga no barramento.

Como é 8 bits, há **overflow** (dá a volta): `200 + 100 = 44` (`300 - 256`).

7. Registrador de Saída — `output_register.v`

O que é: guarda o **resultado** para mostrar nos LEDs/displays. É o único jeito do processador "falar" com o mundo.

```
always @(posedge clk or negedge clr_bar) begin
    if (!clr_bar)    out_port <= 8'b0;
    else if (!lo_bar) out_port <= bus_in;    // ~Lo ativo -> carrega saída
end
```

- `lo_bar` (~Lo): quando 0, carrega o valor (só na instrução `OUT`).
- Entre dois `OUT`, ele **congela** — por isso, na onda, `out_port` fica parado até o próximo `OUT`.

Papel: no `OUT`, o acumulador é copiado para cá (A → barramento → saída), e o valor aparece no visor.

8. RAM 16×8 — `ram_16x8.v`

O que é: a **memória** — 16 posições de 8 bits, guardando programa e dados. Só leitura (o SAP-1 não tem `STA`).

```
reg [7:0] mem [0:15];           // 16 gavetas de 8 bits
initial begin ... end           // grava o programa (vira ROM na FPGA)
assign data_out = mem[addr];     // leitura combinacional
```

- `addr` (4 bits, vem do MAR): qual posição ler.

- **data_out** (8 bits): o conteúdo, que vai para o barramento (via **~CE**).
- Cada palavra é **[opcode(4) | operando(4)]** se for instrução, ou um número puro se for dado.

Mapa: posições **0-10** = programa; **11-15** = dados. Ex.: **mem[11] = 3**, por isso **LDA 11** carrega 3.

9. Decodificadores de display — **seg7.v** e **seg7_instr.v**

seg7 — converte um valor de 4 bits (0-F) nos 7 segmentos do display (ativo em baixo: 0 acende o segmento).

```
case (val)
  4'h0: seg = 7'b1000000; // acende todos menos o 'g' -> "0"
  ...
endcase
```

seg7_instr — igual, mas converte o **opcode** numa **letra** para mostrar a instrução: **LDA→L**, **ADD→A**, **SUB→5**, **OUT→o**, **HLT→H**.

Esses dois só existem para a **placa** (não afetam a lógica do processador).

10. Blocos de placa — **clock_divider.v** e **debouncer.v**

Só entram no topo de FPGA (**sap1_fpga.v**), não no núcleo:

- **clock_divider**: divide os 50 MHz da placa para **~1 Hz**, para você ver a execução acontecer devagar (**freq = 50MHz / (2·DIV)**).
- **debouncer**: filtra o "ruído" mecânico do botão, para cada apertado do **KEY[1]** gerar **um** pulso limpo de clock (modo passo a passo).

Tabela-resumo dos sinais de controle

Bloco	Sinal de carga/ação	Ativo em	Quando age
PC	cp (Cp)	alto	T2 (incrementa)
MAR	lm_bar (~Lm)	baixo	T1 e T4
RAM	ce_bar (~CE)	baixo	T3 e T5 (lê)
IR	li_bar (~Li)	baixo	T3
A	la_bar (~La)	baixo	T5 (LDA), T6 (ADD/SUB)
B	lb_bar (~Lb)	baixo	T5 (ADD/SUB)
ALU	su (Su) / eu (Eu)	alto	T6
Saída	lo_bar (~Lo)	baixo	T4 (OUT)

O que **decide** ligar cada sinal em cada estado é o **controlador/sequenciador** — explicado em `FSM_explicacao.md`.